

VulSCA: A Community-Level SCA Approach for Accurate C/C++ Supply Chain Vulnerability Analysis

Yutao Hu, Chaofan Li, Yueming Wu, Yifeng Cai, Deqing Zou

NDSS 2026

DaEun Kim

August 28th, 2026

(SSP Intern Seminar)

Background

Why C/C++ Supply Chain Analysis?

- **TPLs(Third-Party libraries)** are widely used in C/C++ projects
- Dependency management is not standardized
- Direct source-code reuse complicates SCA
- Limitations of Existing approaches

➔ Need accurate and efficient supply chain vulnerability analysis

➔ **VulSCA: Community-level SCA**

Existing Approaches

Version-based

- Dependency-based detection
- Identifies reused TPL/version
- Flags vulnerabilities associated with detected versions

- ✓ Dependency detection
- X Vulnerable code existence
- X Reachability analysis

Code-based

- Clone-based detection
- Matches known vulnerable code/functions
- Requires repository-wide scanning

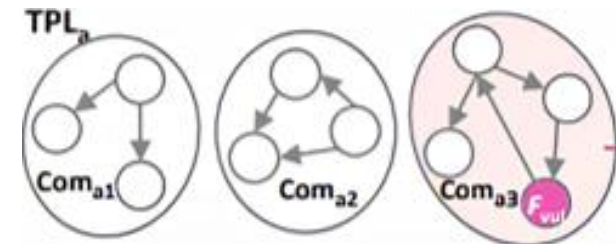
- X Dependency detection
- ✓ Vulnerable code existence
- X Reachability analysis

Key Idea

Community-level SCA

- Model the **call graph as a social network**
- **Semantic community**: tightly connected functions representing a functional module
- Analyze TPL reuse and vulnerabilities at the **community level**

- ✓ Dependency detection
- ✓ Vulnerable code existence
- ✓ Reachability analysis



VulSCA Overview

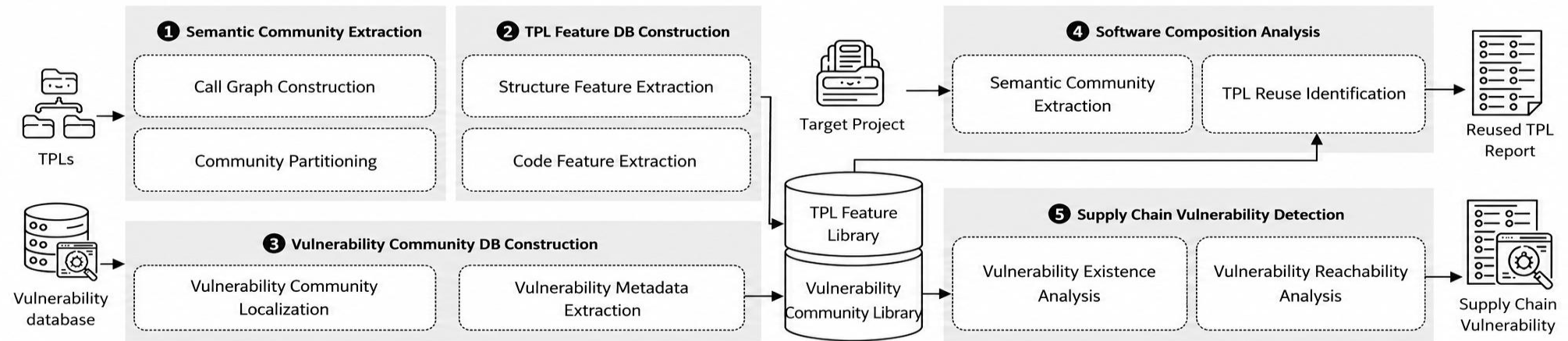


Fig. 2. High-level Workflow of *VulSCA*

Methodology

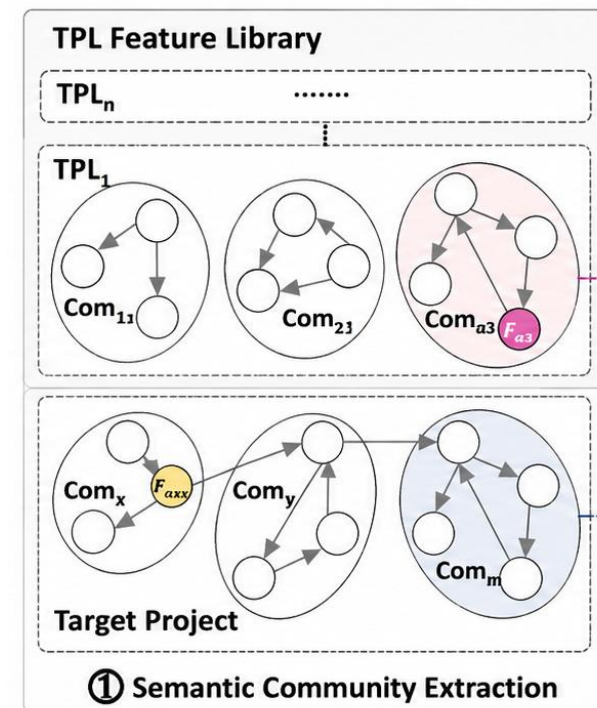
Community-Level SCA

1. Semantic Community Extraction

- Function call graph
- Duplicate functions → **virtual nodes**
- **Infomap** → Semantic communities partitioning

2. TPL feature DB construction

- Community features: Structural V_s + Code V_c
- V_s : 13 call-graph properties features
- V_c : token-level code features



Methodology

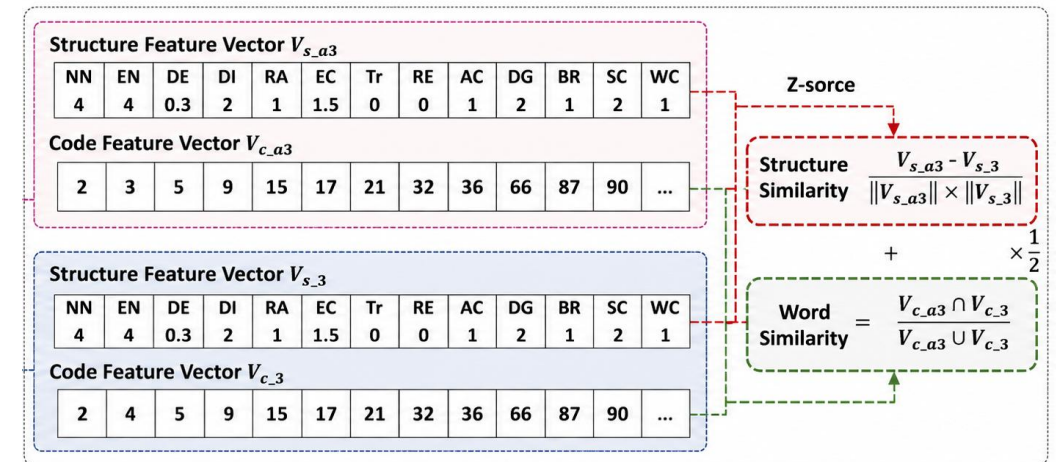
Community-Level SCA

3. Vulnerability community DB construction

- Vulnerable functions → TPL semantic communities
- Community ID → vulnerable function hash set

4. Software Composition Analysis

- V_s : cosine similarity / V_c : Jaccard similarity
 - $V_s < 0.5 \rightarrow$ skip V_c
 - **similarity $\geq \theta$** → reuse



Methodology

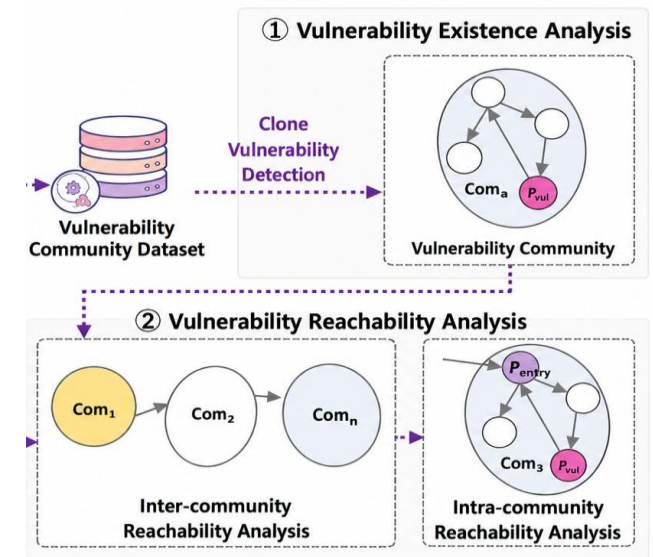
Vulnerability Existence + Reachability Analysis

5-1. Vulnerability Existence Analysis

- **Reused communities** → Vulnerability Community DB
- Clone detection

5-2. Vulnerability Reachability Analysis

- **Inter-community**: Entry community → Vulnerability community
- **Intra-community**: Entry function → Vulnerable function



Evaluation

RQ1: Parameter Selection

- **Community detection algorithm**
 - Infomap → highest SCA F1-score
- **Community similarity threshold θ**
 - Stable SCA performance at 0.83-0.97
 - $\theta = 0.88$

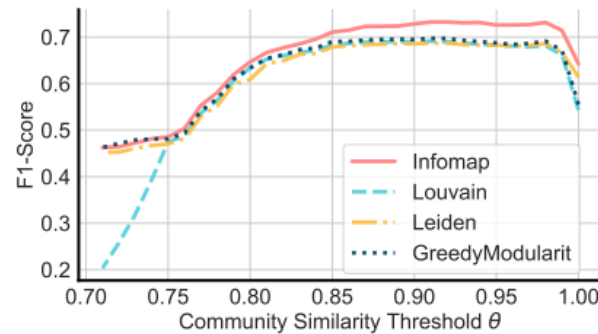


Fig. 5. SCA Results of *VulSCA* under Varying Community Detection Methods

Evaluation

RQ2: SCA Accuracy

- **Evaluation Dataset**

- 80 valid projects / 484 TPLs

- **FP / FN**

- FP: nested dependency / similar communities
- FN: Incomplete call graph / nested dependency

TABLE V
COMPARISON OF SCA EFFECTIVENESS AND EFFICIENCY BETWEEN
CENTRIS, *OSSFDP*, AND *VulSCA*

SCA Method	TP	FP	Precision	Recall	F1-Score	AUPRC
<i>CENTRIS</i>	538	401	0.57	0.62	0.60	-
<i>OSSFDP</i>	715	520	0.58	0.83	0.68	0.582
<i>VulSCA</i>	588	176	0.77	0.68	0.72	0.634

Evaluation

RQ3: Vulnerability Detection

- **Comparison with existing methods**
 - **Version-based:** F1-scores ↑ 44-48%
 - **Code-based:** F1-scores ↑ 17-23%
 - **Hybrid:** F1-scores ↑ 7-19%
- **FP / FN**
 - FP: Reachability limitation
 - FN: Missed TPL reuse / Limited clone detection / Incomplete call graph

TABLE VII
COMPARISON OF VULNERABILITY DETECTION EFFECTIVENESS BETWEEN
VULSCA AND OTHER DETECTION SOLUTIONS

Group	Method	FP	TP	F1	Precise	Recall
Version-based	<i>OSSFDP</i>	560	22	0.07	0.04	0.42
	<i>CENTRIS</i>	310	22	0.11	0.07	0.42
Code-based	<i>Vuddy</i>	62	30	0.41	0.33	0.57
	<i>Fire</i>	113	34	0.34	0.23	0.64
	<i>AntMan</i>	75	30	0.38	0.29	0.57
Code-based Combined with Version-based	<i>OSSFDP-Hash</i>	35	28	0.48	0.44	0.53
	<i>OSSFDP-Vuddy</i>	41	28	0.46	0.41	0.53
	<i>OSSFDP-Fire</i>	87	33	0.38	0.28	0.62
	<i>OSSFDP-AntMan</i>	44	29	0.46	0.40	0.55
	<i>VIScan</i>	46	17	0.29	0.27	0.32
<i>VulSCA</i>	<i>VulSCA_{hash} (default)</i>	13	25	0.55	0.66	0.47
	<i>VulSCA_{Vuddy}</i>	13	27	0.58	0.68	0.51
	<i>VulSCA_{Fire}</i>	25	31	0.57	0.55	0.58
	<i>VulSCA_{AntMan}</i>	20	28	0.55	0.58	0.53

Evaluation

RQ4: Efficiency

- **VulSCA-Hash**: 91 s/project
- **Comparison Units**: 3,522 functions → 918 communities
- **Peak Memory**: 16.2 GB

TABLE IX
COMPARISON OF EFFICIENCY BETWEEN *VulSCA* AND OTHER DETECTION SOLUTIONS

Group	Method	Software Composition Analysis Time (s)		Vulnerability Detection Time (s)		Total Time (s)	Avg. Time (s)	Peak Memory Usage (GB)	Average Memory Usage (GB)
		Feature Extraction Time (s)	TPL Identification Time (s)	Detection Time (s)	Reachability Analysis Time (s)				
Version-based	<i>OSSFDP</i>	6,382	12	1	-	6,395	80	10.2	4.5
	<i>CENTRIS</i>	6,588	16,579	1	-	23,168	290	11.9	5.2
Code-based	<i>Vuddy</i>	8,027	-	3,440	-	11,467	143	9.9	6.9
	<i>Fire</i>	2,464	-	9,856	-	12,320	154	10.1	7.8
	<i>AntMan</i>	19,553	-	13,035	-	32,588	407	7.9	6.6
Code-based	<i>OSSFDP-Hash</i>	6,376	12	1	-	6,388	80	10.2	4.3
	<i>OSSFDP-Vuddy</i>	6,762	12	166	-	6,939	87	10.2	4.5
Combined with	<i>OSSFDP-Fire</i>	6,775	12	1,601	-	8,388	105	10.2	5.1
Version-based	<i>OSSFDP-AntMan</i>	7,316	12	627	-	7,956	99	10.2	4.7
	<i>VIscan</i>	6,588	16,579	2,385	-	25,544	319	11.9	4.7
VulSCA	<i>VulSCA_{Hash}</i> (default)	7,011	46	1	182	7,240	91	16.2	7.2
	<i>VulSCA_{Vuddy}</i>	7,162	46	351	189	7,748	97	16.2	7.1
	<i>VulSCA_{Fire}</i>	7,375	46	1,455	310	9,187	115	16.2	7.3
	<i>VulSCA_{Antman}</i>	7,867	46	570	214	8,697	109	16.2	7.1

Limitations

- **Incomplete call graph construction**
 - Affects community partitioning and reachability
- **Limited clone detector capability**
 - May miss complex/modified vulnerable clones

Conclusion

- **VulSCA : Community-level SCA framework**
- Combines dependency identification + vulnerability existence detection + reachability analysis
- SCA F1-score ↑ 4-12%
- vulnerability detection F1-score ↑ 7-48%
- Identified 32 previously unpatched vulnerabilities

Thank you for listening.

Q & A

Appendix

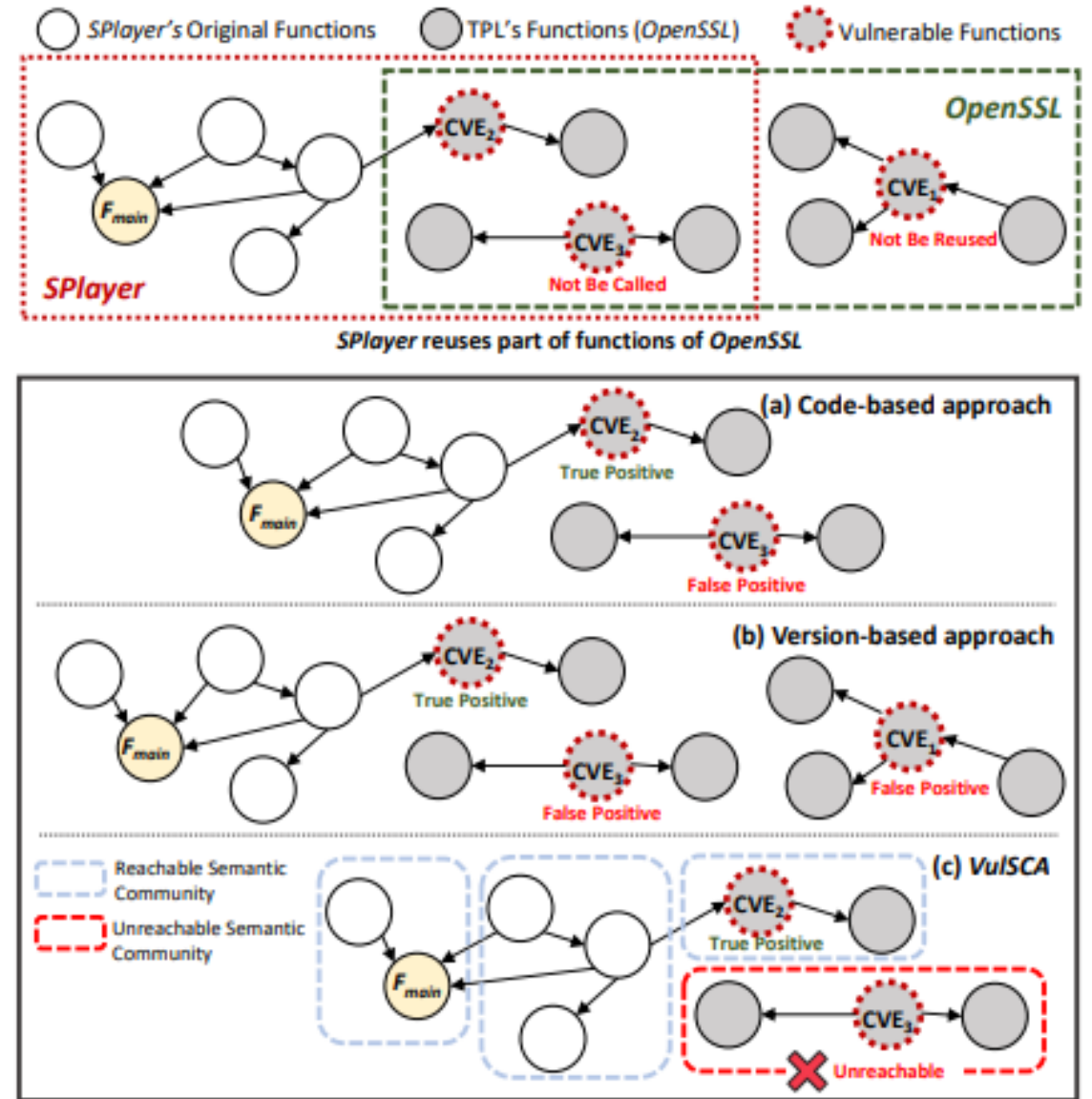


Fig. 1. An Example of *SPlayer* Reusing *OpenSSL*

Appendix – structural feature extraction

TABLE II
DETAILS OF COMMUNITY STRUCTURE FEATURES

No.	Social Network Property	Abbr.	Description	Feature Processing Time (s)
1	Number of Nodes	NN	The number of nodes in the network	<0.001
2	Number of Edges	NE	The number of edges in the network	<0.001
3	Density	DE	The density of the network	<0.001
4	Diameter	DI	The maximum eccentricity of the network	<0.001
5	Radius	RA	The minimum eccentricity of the network	<0.001
6	Eccentricity	EC	The maximum distance from a node to other nodes	<0.001
7	Reciprocity	RE	The ratio of bidirectional edges to all edges	<0.001
8	Algebraic Connectivity	AC	The second smallest eigenvalue of the <i>Laplacian</i> matrix	<0.001
9	Degree	DG	The number of edges connected to a node	<0.001
10	Bridges	BR	The number of bridges in the network	<0.001
11	Transitivity	TR	The fraction of closed triangles in the graph	<0.001
12	Strongly Connected Components	SC	The number of strongly connected components in the network	0.009
13	Weakly Connected Components	WC	The number of weakly connected components in the network	0.007